Infrastructure as Code with Copilot

Prompt it, check it, scan it

Session M4-02 | 2 hours 30 minutes

Agenda

Segment	Time
What IaC is and why	8 min
Terraform in plain words	12 min
Networking terms you need	5 min
Prompting Copilot for IaC	12 min
Where Copilot is weak	8 min
Demo: prompt, plan, scan	10 min
Bicep, pros and cons, handoff	5 min

What IaC is

Infrastructure as code: you describe servers, networks, and storage in files. A tool builds them.

- **Review it:** infrastructure changes go through pull requests.
- **Repeat it:** the same files give the same setup, every time.
- **Undo it:** git history is your change log.
- **Scan it:** tools read the files before anything exists.

Clicking in a portal gives you none of these.

Terraform in plain words

Word	Means
Resource	One thing to create: a VPC, a bucket
Variable	An input, so values aren't hardcoded
Output	A value the code shows or passes on
Module	A folder of resources you call like a function
Provider	The plugin that talks to a cloud (AWS, Azure)
State	Terraform's record of what it built
Plan	A preview of changes. Nothing changes yet.

The workflow

```
fmt → init → validate → plan → (review) → apply
```

Command	Does
`fmt`	Fixes style
`init`	Downloads providers, loads modules
`validate`	Checks syntax and references. No cloud call.
`plan`	Shows what would change
`apply`	Changes the cloud. **Not in this lab.**

State, in one slide

Terraform keeps a **state file**: what it built, and the IDs.

- `plan` compares your code with state, then proposes the difference.
- State can contain **secrets**. Don't commit it.
- Teams store it remotely, with locking.
- Lose it and Terraform no longer knows what it owns.

Networking terms

- **VPC:** your private network in AWS.
- **CIDR:** an address range, like `10.0.0.0/16`.
- **Subnet:** a slice of the VPC, in one zone.
- **Private subnet:** no direct route from the internet.
- **Security group:** a firewall for resources. Rules say who may connect, on which port.
- **`0.0.0.0/0`:** every address on the internet.

Our lab architecture

```
VPC 10.0.0.0/16
├── private subnet 10.0.0.0/24  (zone a)
├── private subnet 10.0.1.0/24  (zone b)
└── S3 bucket: private, versioned, encrypted
```

Small on purpose. You read all of it.

Prompting Copilot for IaC

Give it **five things**:

1. **Architecture:** what exists and how it connects
2. **Constraints:** private, encrypted, no public access
3. **Naming and tags:** `${var.name_prefix}-<purpose>`
4. **Variables, not literals:** "hardcode nothing that belongs in a variable"
5. **Limits:** "no apply", "only arguments in provider 5.x"

Then **read the plan**. The plan is the review point.

Weak and strong

Weak

```
Make me a VPC and a bucket in AWS.
```

Strong

```
Module modules/baseline: one VPC from var.vpc_cidr, two private subnets
from cidrsubnet, one bucket with public access blocked, versioning, and
encryption. Name ${var.name_prefix}-<purpose>. Tag with var.tags.
No hardcoded values. Do not run apply.
```

Put the rules in instructions

Repeat yourself once, in `.github/copilot-instructions.md`.

```
- Use hashicorp/aws ~> 5.0.
- Variables need a description and a type.
- Never write secrets into .tf files.
- Do not run terraform apply.
```

Copilot reads it on every request. The lab starter has one.

Where Copilot is weak

Problem	Example
Invented arguments	A resource argument that doesn't exist
Old versions	Pins an old provider; uses retired bucket syntax
Insecure defaults	`0.0.0.0/0`, public buckets, no encryption
Hardcoded values	CIDRs and names inline
Plausible but wrong	Valid code, wrong architecture

`validate` catches the first. Scanning catches the third. **You** catch the rest.

Scan before you plan

Trivy reads Terraform files and reports misconfigurations. No cloud account.

```
trivy config --severity HIGH,CRITICAL .
```

```
AWS-0107 (HIGH): Security group rule allows unrestricted
ingress from any IP address.
```

Alternatives: Checkov, TFLint (a linter for provider-specific mistakes). Pick one for CI.

Demo

1. Prompt Copilot with the five parts.
2. `fmt`, `init`, `validate`, `plan`.
3. Ask for "SSH from anywhere".
4. `trivy config`. Read the finding.
5. Fix with a variable. Scan again.

Bicep in one slide

Azure's IaC language. Same ideas, different words.

Terraform	Bicep
`variable`	`param`
`resource`	`resource`
`output`	`output`
Provider and state	Azure handles both

```
az bicep build --file main.bicep
```

Pros and cons

Pros	Cons
Fast first draft	Confident wrong answers
Knows the syntax	Outdated versions
Explains plans and errors	Insecure shortcuts
Converts between tools	Doesn't know your org's rules

Tip: let Copilot draft. Let `plan` and a scanner judge.

Common mistakes

- Prompting for "a VPC" with no constraints
- Accepting the code without reading the plan
- Committing `.terraform/`, state files, or plans
- "Fixing" `validate` by deleting the failing line
- Treating a clean scan as proof of a safe design

Lab

One module, one scan, one Bicep file. 90 minutes, no cloud account.